

Jour 02 / 05

Selenium.

Quand la page se construit côté JavaScript,
il faut piloter un vrai navigateur — pas juste un client HTTP.

Planning de la journée

7 heures, 4 phases.



LIVRABLE FIN DE JOUR · deux scrapers Selenium opérationnels, mode headless configuré, screenshots automatiques en cas d'échec.

Module 1 · Diagnostic

Pourquoi quitter requests.

LE SYMPTÔME

Un HTML quasi vide

- `requests.get(url).text` ne contient que la coquille HTML et quelques `<script>`.
- Les données sont injectées **après** exécution du JavaScript.
- Cas typiques : SPA React/Vue, contenus lazy-loadés, connexion utilisateur, paywalls.

LA SOLUTION

Un vrai navigateur

- Selenium pilote Chrome, Firefox, Edge ou Safari via le protocole **WebDriver**.
- Le JS s'exécute, le DOM se peuple, on lit le résultat final.
- On peut **interagir** : clic, saisie, scroll, attente conditionnelle.

Diagnostic · 30 secondes

Statique ou dynamique ?

CRITÈRE	Statique → requests + BS4	Dynamique → Selenium
Source du HTML	Tout est dans la réponse serveur	HTML final construit par JS
Test rapide	curl URL → données visibles	curl URL → coquille vide
Vitesse	~50 ms / page	~2 à 5 s / page
Ressources	Quelques Mo de RAM	Navigateur complet — 200 Mo+
Exemples	Blog du Modérateur, Wikipédia	Doctolib, LinkedIn, Twitter

Architecture · WebDriver

Qui parle à qui ?

1 · VOTRE SCRIPT

Code Python

Bindings selenium. Vous appelez
`driver.get(url), .find_element(...)`.

```
pip install selenium
```



2 · WEBDRIVER

Pont protocole

Exécutable spécifique au navigateur. Traduit les ordres Selenium en commandes natives.

```
chromedriver / geckodriver
```



3 · NAVIGATEUR

Chrome & co.

L'instance réelle. Charge la page, exécute le JS, fait le rendu. Peut être **headless**.

```
Chrome / Firefox / Edge
```

→ Depuis Selenium 4.6, Selenium Manager télécharge automatiquement le bon driver. Plus besoin de gérer la version à la main.

Setup · Selenium 4

Installer Selenium.

01 Selenium 4.10+

Inclut Selenium Manager. Plus de driver à installer manuellement.

02 Un navigateur installé

Chrome ou Firefox. Versions à jour, sinon le driver refusera.

03 webdriver-manager (option)

Solution alternative si Manager intégré ne convient pas.

04 Test « hello world »

Ouvrir `example.com` et vérifier le titre.

```
# 1. Installation pip install selenium# 2. Hello worldfrom selenium import webdriver from selenium.webdriver.chrome.options import Options opts =Options() opts.add_argument("--headless=new") opts.add_argument("--window-size=1920,1080") driver = webdriver.Chrome(options=opts) try: driver.get("https://example.com") print(driver.title) # Example Domainfinally: driver.quit()
```

Module 2 · Pratique

Premier scraper complet.

```
from selenium import webdriver from selenium.webdriver.common.by import By from
selenium.webdriver.support.ui import WebDriverWait from selenium.webdriver.support import
expected_conditions as EC defscrape_quotes(): driver = webdriver.Chrome() try:
driver.get("https://quotes.toscrape.com/js/") # Attendre que les citations soient injectées par JS
WebDriverWait(driver, 10).until( EC.presence_of_all_elements_located( (By.CSS_SELECTOR, "div.quote") ) )
quotes = driver.find_elements(By.CSS_SELECTOR, "div.quote") results = [ { "texte":
q.find_element(By.CSS_SELECTOR, ".text").text, "auteur": q.find_element(By.CSS_SELECTOR, ".author").text, }
for q in quotes ] return results finally: driver.quit() if __name__ == "__main__": for q inscrape_quotes():
print(f"{q['auteur']} → {q['texte'][:60]}")
```

À DÉCRYPTER

try/finally : garantir la fermeture du navigateur même en cas d'erreur.

WebDriverWait : ne pas lire le DOM avant que le JS l'ait peuplé.

By.CSS_SELECTOR : mêmes sélecteurs qu'avec BS4.

.text vs .get_attribute : texte visible vs attribut HTML.

Stratégies de sélection

6 façons de viser un nœud.

STRATÉGIE	EXEMPLE	QUAND L'UTILISER
By.ID	"login-form"	L'identifiant existe et est stable
By.CSS_SELECTOR	".result > h3 a"	Choix par défaut. Lisible, rapide.
By.XPATH	"//button[text()='Accepter']"	Quand il faut filtrer par texte ou parent
By.NAME	"email"	Champs de formulaire
By.LINK_TEXT	"En savoir plus"	Lien identifié par son libellé exact
By.TAG_NAME	"article"	Quand on veut TOUS les éléments du type

Simuler l'utilisateur

Clic, saisie, scroll.

```
from selenium.webdriver.common.keys import Keys from selenium.webdriver.support.ui import Select #
1. Saisir du texte champ = driver.find_element(By.NAME, "q") champ.clear() champ.send_keys("web
scraping") champ.send_keys(Keys.ENTER) # 2. Cliquer sur un bouton btn =
driver.find_element(By.CSS_SELECTOR, "button.search") btn.click() # 3. Sélectionner dans un <select>
Select(driver.find_element(By.NAME, "ville")) \ .select_by_visible_text("Paris") # 4. Scroll infini
: descendre, attendre, recommencer while True: h_before = driver.execute_script("return
document.body.scrollHeight") driver.execute_script("window.scrollTo(0,
document.body.scrollHeight);") time.sleep(2) if driver.execute_script("return
document.body.scrollHeight") == h_before: break
```

RÉFLEXES

→ **clear()** avant `send_keys()` sur un champ qui peut être pré-rempli.

→ **Click intercepted ?** Faire un `scrollIntoView` d'abord.

→ **execute_script()** est l'arme ultime pour interagir avec le JS de la page.

→ **Screenshot** avant chaque échec pour déboguer visuellement.

Synchronisation

time.sleep() est une dette technique.

✗ À ÉVITER · attente fixe

```
import time driver.get(url) time.sleep(5) # on espère que ça
suffit... btn = driver.find_element(By.ID, "go") btn.click()
```

- Trop court : NoSuchElementException aléatoire selon la charge réseau.
- Trop long : 5 s par page × 1000 pages = 1h20 perdues.
- **L'attente fixe force à choisir entre flakyness et lenteur.**

✓ BONNE PRATIQUE · attente conditionnelle

```
wait = WebDriverWait(driver, 10) driver.get(url) btn = wait.until(
EC.element_to_be_clickable( (By.ID, "go") ) ) btn.click()
```

- Le code reprend dès que la condition est remplie.
- Échec rapide et net si la condition n'arrive pas en 10 s.
- **Plus rapide ET plus fiable que time.sleep().**

Friction du Web moderne

Bannières et popups.

Un scraper se prend une bannière cookies à la 1re visite. **3 stratégies, classées par robustesse.**

STRATÉGIE 1

Cliquer "Accepter"

```
btn = wait.until( EC.element_to_be_clickable(
    (By.XPATH, "//button[text()='Accepter']" ) ) )
btn.click()
```

Simple. Mais le libellé change selon les sites.

STRATÉGIE 2

Injecter le cookie

```
driver.get(url) driver.add_cookie({ "name":
    "consent", "value": "accepted", "domain":
    ".doctolib.fr", }) driver.refresh()
```

Plus rapide. Demande de connaître le nom du cookie.

STRATÉGIE 3

Profil persistant

```
opts.add_argument( "--user-data-dir="
    "./chrome-profile" ) # le profil retient le #
consentement entre runs
```

La plus robuste. Demande un peu de gestion de fichiers.

Doctolib.

Cible : www.doctolib.fr — collecter les créneaux disponibles d'une spécialité dans une ville donnée.

MISSION

Pour chaque praticien...

- ① **nom & spécialité**
- ② **adresse** du cabinet
- ③ **type de consultation** (cabinet / vidéo)
- ④ **prochains créneaux** (3 max)
- ⑤ **URL** de la fiche

PIÈGES À PRÉVOIR

Le code doit...

- ✓ Gérer la bannière de cookies
- ✓ `WebDriverWait` sur la liste de résultats
- ✓ Scroller pour charger les praticiens suivants
- ✓ Screenshot en cas d'échec d'élément
- ✓ Délai aléatoire entre les visites de fiches

Les Echos.

Cible : www.lesechos.fr — collecter les titres à la une, les rubriques, les chapôs et les dates.

MISSION

Pour chaque article

- ① **titre** en une
- ② **rubrique** (Finance, Tech, ...)
- ③ **chapô** visible sans paywall
- ④ **heure** de publication
- ⑤ **flag premium** (abonnés / libre)

OBJECTIFS BONUS

Aller plus loin

- ✓ Lancer en mode `--headless=new`
- ✓ Comparer le temps avec / sans headless
- ✓ Tester si `requests` seul suffirait (view-source)
- ✓ Documenter la décision dans le README

Production

Headless & performance.

```
from selenium.webdriver.chrome.options import Options
opts = Options() # 1. Headless mode moderne (Chrome 109+)
opts.add_argument("--headless=new") # 2. Taille de fenetre fixe — beaucoup de sites adaptent leur DOM
opts.add_argument("--window-size=1920,1080") # 3. Bloquer les ressources inutiles : images, notifications
prefs = { "profile.managed_default_content_settings.images": 2,
          "profile.default_content_setting_values.notifications": 2, }
opts.add_experimental_option("prefs", prefs) # 4. Désactiver l'auto-fill et autres bruits
opts.add_argument("--disable-extensions")
opts.add_argument("--no-sandbox") # utile en Docker
opts.add_argument("--disable-dev-shm-usage") # 5.
Timeout du chargement de page
driver = webdriver.Chrome(options=opts)
driver.set_page_load_timeout(30)
```

EFFET MESURÉ

Headless	-30 %
Sans images	-50 %
eager load	-15 %
Cumul	≈ ÷3

→ **Mode visible en dev**, headless en prod. Le debug est 10× plus rapide à l'écran.

Friction · côté serveur

On vous a vu venir.

Les sites lisent une dizaine de signaux pour décider si vous êtes un bot. **Voici les 4 qui comptent.**

01 navigator.webdriver

Variable JS qui vaut true en Selenium. Masquable via `excludeSwitches: ["enable-automation"]`.

02 Empreinte HTTP / TLS

L'ordre des headers, le ciphering TLS trahissent requests Python. Selenium est plus crédible (vrai Chrome).

03 Rythme & pattern

Une requête toutes les 200 ms = bot. Espacer, randomiser, faire des pauses réalistes.

04 Cloudflare / Datadome

Solutions WAF avec challenges JS et CAPTCHA. **Si déclenché, arrêter et repenser la question légale.**

→ Outils comme `undetected-chromedriver` ou `selenium-stealth` existent. **À utiliser uniquement dans un cadre légal explicite.**

Lucidité technique

Selenium n'est pas la réponse universelle.

Quatre alternatives à considérer **avant** de lancer un navigateur.

ALTERNATIVE 1

API officielle

Inspecter l'onglet Network du DevTools : 90 % des SPA exposent une API JSON. Plus rapide, plus stable.

ALTERNATIVE 2

Playwright

Le successeur moderne. API plus simple, attentes auto, multi-onglets natif. À évaluer en projet réel.

ALTERNATIVE 3

Scrapy + Splash

Pour un crawl massif, Scrapy avec un renderer JS scale mieux. Vu demain (Jour 3).

ALTERNATIVE 4

Flux RSS / Atom

Sur les sites éditoriaux, le flux RSS livre déjà la donnée structurée. **Ne pas oublier !**

Synthèse

Ce qu'on retient.

01 Selenium n'est utile que si la page se construit en JS.

Sinon requests + BS4 reste 30× plus rapide. Toujours faire le test `curl` avant.

02 WebDriverWait à la place de `time.sleep()`. Toujours.

Le seul moyen d'avoir un scraper à la fois rapide et fiable.

03 Headless + blocage des images = ÷3 sur la durée de scrape.

Garder le mode visible pour debug, le headless pour les runs.

DEMAIN — JOUR 03 · Scrapy. Quand il faut crawler des milliers de pages avec retry, throttling et pipelines.